

COURSE 2 OF 6 · EXPERT-DRIVEN DEVELOPMENT

Builder Orientation

From AI user to AI-assisted builder. Two hours, mostly hands-on, with one rule: decompose before you build.

DURATION

2 hours

AUDIENCE

Aspiring builders

PREREQUISITE

**AI Fluency
Fundamentals**

OUTCOME

**A working prototype +
a real plan**

SPEAKER NOTES

OPEN Welcome students by name as they join the Teams meeting. Share your screen now and put this slide up before the start time hits.

Say (60 seconds): “Welcome to Builder Orientation. This is Course 2 of 6. Course 1 was about using AI well. Today is about **building** with it. Two hours. Most of it is hands-on. By the end you will have a working prototype, even if it is rough, and a real plan you will execute before Course 3.”

Set the contract: “This is not a lecture. There are about 30 minutes of me talking spread across two hours. The other 90 minutes is you typing, decomposing on paper, and reviewing each other’s work. Cameras on if you can. Mics off unless you have a question.”

Verify prerequisite: Ask the room: “Quick check — raise a hand or type in chat if you have **not** completed AI Fluency Fundamentals.” If anyone hasn’t, note them privately and pair them with a stronger partner for Module 3.

Transition: “Before we look at today’s agenda, let me anchor where we left off in Week 1.”

WHERE WE LEFT OFF

Week 1 gave you the management skills. Week 2 puts them on a build.

FROM WEEK 1, YOU SHOULD REMEMBER

- The six 201 skills — Context, Quality, Decomposition, Iteration, Workflow, Frontier
- The Delegation Equation — your time vs. AI's success rate vs. review cost
- Centaur (clear handoffs) and Cyborg (continuous loop)
- The jagged frontier — AI is uneven, and the boundary is not intuitive

WHAT CHANGES TODAY

- You stop managing one conversation. You start managing a project.
- Decomposition graduates from a prompt skill to a system-design skill.
- You meet your first errors. You debug them on purpose.
- You leave with a real prototype, or a real plan to build one.

SPEAKER NOTES

RECAP Spend 60–90 seconds here. Don't re-teach Week 1 — just **name** the anchors so the room remembers them.

Say: “Show of hands — how many of you have used an AI tool at work since Week 1?” Note who is active and who is not. Active builders will lead Module 3.

Then: “Today we move from being good **users** to being functional **builders**. The skills are the same. The scope is bigger. Instead of one conversation, you are managing a whole project through AI.”

Frontier reminder: “Building has its own frontier. Some things AI builds beautifully. Some things break in interesting ways. We will see both today.”

Transition: “Here is the next two hours.”

AGENDA · 120 MINUTES

Five modules. Mostly hands-on.

- **0:00 - 0:15** **Module 1** From User to Builder 15 MIN · TALKING
- **0:15 - 0:40** **Module 2** Live Build — Equipment Tracker **25 MIN · LIVE BUILD**
- **0:40 - 0:50** **Break** 10 MIN
- **0:50 - 1:30** **Module 3** Student Build + When Something Breaks **40 MIN · HANDS-ON**
- **1:30 - 1:50** **Module 4** Decomposition Framework **20 MIN · EXERCISE**
- **1:50 - 2:00** **Module 5** Wrap-Up & Assignment 10 MIN · TALKING

SPEAKER NOTES

PACING Read the agenda quickly. Emphasise the ratio.

Say: “Out of 120 minutes, about 30 minutes is me talking. The other 90 minutes is you working — with me on a live build, on your own, with a partner, and with paper. Plan accordingly. Have your laptop ready, have a piece of paper ready, and have GenAI.mil or your AI tool open in a tab.”

Tooling check: “Take 15 seconds right now — open GenAI.mil or your approved AI tool. If you can’t reach it, drop a note in chat now, not in 40 minutes when we start the live build.”

Transition: “Let’s start with the only theory section we have today.” Click forward.

From user to builder.

15 minutes · talking + room poll

201 SKILL: TASK DECOMPOSITION

SPEAKER NOTES

SECTION DIVIDER Pause for one beat on this slide so it lands. Don't read the title aloud.

Say: "Module 1. Fifteen minutes. This is the only stretch of the day where I will talk uninterrupted. Lock in."

Building is the same skills, at a bigger scope.

AS A USER

You manage one conversation.

One prompt. One iteration loop. One output.
You judge if it's good and move on.

AS A BUILDER

You manage a project through AI.

Many prompts, in a sequence you designed. AI generates code; you decide if it works in a system. The bottleneck is your decomposition, not AI's talent.

A user asks AI to draft an email. A builder asks AI to create the system that drafts, tracks, and sends emails. The difference is decomposition.

SPEAKER NOTES

ANCHOR This is the central idea of the whole course. Don't rush.

Say: "In Week 1 you managed one conversation. You typed something, you got something back, you judged it, you iterated. Today you do the same thing — but in **sequence**, across an entire project. The skills don't change. The scope does."

Drive the point home: "A user asks AI to draft an email. A builder asks AI to create the system that drafts, tracks, and sends emails. Same skills. Different scope. The difference is not talent. It's decomposition."

Avoid: Don't romanticise 'builders.' Most students in this room manage Marines. They already break work into pieces every day. We're just applying that to AI.

“Before you open any tool — before you type a single prompt — decompose on paper.

THE BUILDER MINDSET • COURSE 2, MODULE 1

SPEAKER NOTES

BIG IDEA Hold this slide for a beat. It’s the tattoo for the day.

Say: “If you remember nothing else from today, remember this. Decompose **first**. On paper. With nobody’s tool open. The number-one mistake new builders make is opening the tool first and trying to figure it out as they go. That works for simple things. It fails completely for anything with three or more moving parts.”

Ask the room: “Anyone here ever tried to build a Power App by just opening Power Apps and clicking around?” (You’ll get a laugh and a few hands.) “How did that go?”

Transition: “The reason decomposition matters so much is that the quality of your plan predicts the quality of your build. A vague plan produces a vague tool. A specific plan produces a specific tool.”

Quick check — name three of the six 201 skills.

01

Context Assembly

What information to provide and why.

02

Quality Judgment

When to trust output and when to verify.

03

Task Decomposition

Breaking work into AI-appropriate chunks.

04

Iterative Refinement

Moving from 70% to 95% in passes.

05

Workflow Integration

Embedding AI into how work gets done.

06

Frontier Recognition

Knowing when you're outside AI's capability.

SPEAKER NOTES

ACTIVITY (3 MIN) Cold-call two or three students. Ask each to name a 201 skill from memory **before** revealing the grid — if you flip to this slide too fast they'll just read it.

Say: "Without looking, name three of the six 201 skills." Wait. Pick someone. Then someone else.

If the room can't name 4+: Pause. Do a 2-minute refresher right here. Walk through all six in the grid, point at each card, and read the one-line description. Don't go back to Course 1 — just refresh.

If the room nails it: Move on quickly. "Good. We'll apply Decomposition, Context Assembly, and Iterative Refinement most heavily today. Quality Judgment shows up when we debug. Frontier Recognition shows up when something breaks."

Transition: "Module 1 done. Time to build something. Eyes back on me."

Live build — Equipment Tracker.

25 minutes · you watch, I narrate every decision

LIVE · DECOMPOSE → BUILD → ITERATE

SPEAKER NOTES

SECTION DIVIDER One beat on this slide. Read the duration aloud so people pace themselves.

Say: “Module 2. Twenty-five minutes. I am going to build a real tool live, in front of you, from scratch. You watch. You don’t type along. The value is not the finished app — the value is hearing me narrate every decision.”

Setup check: Confirm GenAI.mil is open in one browser tab and Power Apps (or your live-build target) is open in another. **Do NOT have anything pre-built.** The whole point is that the room watches you make decisions in real time.

“I need to track equipment checkout for my section.”

A rifle section — about 30 Marines, about 50 items. Items get checked out, checked back in, and sometimes go overdue. We need one screen that handles all three.

THREE RULES FOR THE NEXT 25 MINUTES

- I narrate every decision out loud, including the ones I reject.
- You watch. You don't type along. Take notes if you want.
- If something breaks, I debug it live — that's the most useful part.

TOOLING FOR THIS BUILD

- **GenAI.mil** for prompting (preferred for CUI)
- **Power Apps** as the build surface
- **SharePoint** as the data backend
- No pre-built code. We start from blank.

SPEAKER NOTES

FRAME THE BUILD 60 seconds tops on this slide.

Say: “The problem is real. I built one of these for my own section last quarter. It took me about 30 minutes total. Today I'm going to do it in 25 in front of you.”

Set the rules: Read the three rules aloud. Especially: “Don't type along. If you try to follow my prompts in real time you'll fall behind and stop listening to **why** I'm making each choice. Watch first. You get your turn in Module 3.”

Data-handling reminder: “GenAI.mil and CamoGPT are authorised for CUI — anonymise PII unless a PIA covers it. Commercial tools like ChatGPT and Gemini are unclassified-only. We're using fake names today, but the rule sticks.”

Transition: “Step one is not opening the tool. Step one is on a piece of paper.”

Decompose first — on the board, before any tool.

FOUR QUESTIONS

- What data fields do we need?
- What does the user need to do?
- What is the simplest useful version?
- What data structure backs it?

WHY THIS MATTERS

These five minutes on paper are the most valuable five minutes of the build. Bad decomposition means you spend 30 minutes building the wrong thing.

SPEAKER NOTES

ACTIVITY · 5 MIN This is a **group** whiteboard, not a private exercise. Ask the room and write down what they call out. Use a real whiteboard, a Teams whiteboard, or a notepad shared on screen.

Say: “I’m not opening any tool yet. Five minutes on the board first. Tell me — what data fields do we need to track equipment?”

Expected answers (write them up):

- Item name, serial number, assigned to (name), date out, date due, status (out / in / overdue)
- User actions: check out, check in, see what’s out, see what’s overdue
- Simplest version: a form + a list with current status
- Data structure: one row per item, table or SharePoint list

Drive home: “We just spent five minutes on paper. This is the most important five minutes of the whole build. Watch what happens when I open AI now — my prompts are going to be specific because I already know what I want.”

Transition: “Now to AI. The next slide previews the five prompts I’ll use. After that, I switch to the live build.”

Five prompts. In order. Each one builds on the last.

01

Define the problem

Ask for a plan, not code.

02

Data structure

SharePoint list + CSV template.

03

Form + gallery

Auto-app, then refine.

04

Overdue logic

Highlight + check-in button.

05

Polish

Color scheme + summary counts.

Watch the verbs. I never say *“build me an app.”* I say *“give me a plan,” “give me the exact list configuration,” “refine this one piece.”* Each prompt has one job.

SPEAKER NOTES

QUICK MAP 30 seconds on this slide. Don't read each card — just point and call out the verbs.

Say: “You'll see me run roughly five prompts. The first asks for a plan, not code. The second asks for the exact data structure. The third generates the form and the list. The fourth adds the business logic — overdue highlighting and a check-in button. The fifth is polish — color scheme and summary counts.”

Big idea: “Watch what I **don't** do. I never say ‘build me an app.’ That's a 101 prompt and you'll get garbage. I break the work down and feed AI one job at a time.”

Set the parking expectation: “I'm about to switch out of this deck. The next slide is a parking screen so you know where we are while I'm building. When I come back to the deck, we'll debrief what just happened.”

Transition: Click forward to the parking screen, then immediately ALT-TAB to GenAI.mil. Don't leave the parking slide hanging in silence — start narrating Prompt 1 within five seconds.

PRESENTER HAS LEFT THE DECK

Live build in progress.

Watch the presenter's GenAI.mil and Power Apps windows. We'll return to this deck for the debrief in roughly 20 minutes.

WATCHING FOR

How the prompt names a domain (rifle section, ~30 people, ~50 items).

WATCHING FOR

How the presenter rejects an AI suggestion and says *why*.

WATCHING FOR

How the first error gets diagnosed — not panicked over.

DON'T TYPE ALONG · TAKE NOTES · QUESTIONS GO IN CHAT

RETURN CUE: PRESENTER SAYS "BACK TO THE DECK."

SPEAKER NOTES

PARK THE DECK This is the slide that stays up while you're in GenAI.mil and Power Apps. **Switch to your build windows now.**

Pacing across 20 minutes (target):

- **0-3 min — Prompt 1:** Define the problem, ask for a plan. Narrate: "I start with WHO uses this and WHAT platform we have. I ask for a plan, not code."
- **3-7 min — Prompt 2:** Data structure + CSV template. Narrate: "Now I get the exact data structure. I specify field names and types because I know my domain."
- **7-11 min — Prompt 3:** Generate the form + gallery (or use SharePoint Integrate → Power Apps shortcut). Narrate the shortcut.
- **11-16 min — Prompt 4:** Add overdue highlighting and check-in button. **Expect at least one error here** — common ones are Choice column syntax (`Status.Value`) and missing null checks. Debug live, narrate every step.
- **16-19 min — Prompt 5:** Color scheme + summary counts.
- **19-20 min:** Hit ALT-TAB back to the deck and say "Back to the deck."

If something breaks (it should): Don't hide it. Stop, read the error message out loud, narrate the debug prompt you'd send. This **is** the lesson.

If everything works smoothly: Add a stretch — ask AI to add a search box or a date picker. Force one error so the room sees the debug loop.

If the tools fail entirely: Switch to the contingency on the instructor page ("Decomposition Exercise Fallback"). Show the four-bug code block on screen and walk the room through diagnosing it as a class.

Return cue: When you ALT-TAB back to the deck, say **"Back to the deck."** out loud so the room re-anchors.

What just happened.

Three things to name from what you saw — even if my build wasn’t perfect.

DECOMPOSITION FIRST

Five minutes on the whiteboard before any tool. I knew what I wanted before I asked.

PROMPTS HAD ONE JOB

“Give me a plan.” “Give me the data structure.” “Refine this one piece.” Never “build me an app.”

ERRORS WERE NORMAL

When something broke, I read the error, framed a debug prompt, and kept moving. Errors are part of the process.

SPEAKER NOTES

DEBRIEF · 3 MIN This is the “return to deck” landing. Use it to recap what they just watched, then take questions.

Say: “Back to the deck. Three things I want you to name from what you just saw.” Walk through the three cards.

Ask the room: Two questions max —

- “What surprised you about the process?”
- “How long do you think this would have taken before AI?” (Expected answer: days or weeks for a custom app.)

If a student asks how to handle an error you didn’t cover: Defer to Module 3 — they’re about to debug live. “Hold that. You’re going to hit it yourself in 15 minutes.”

Don’t: Don’t walk through your code line by line. The point is the **process**, not the artifact.

Transition: “Take ten. When you come back, you’re building.”

10:00

Stretch. Refill. Open your AI tool. We build at the top of the hour.

SPEAKER NOTES

BREAK · 10 MIN

Before you go silent: “Ten-minute break. When you come back, have GenAI.mil **and** a piece of paper open in front of you. We start Module 3 with you typing, not me.”

During the break: Don’t leave the room. Use the time to (1) confirm everyone has tool access, (2) spot anyone who looked lost in Module 2 and check in privately, (3) reset your own browser tabs so you’re ready to circulate during the build.

Mute: Mute yourself in Teams during the break. Set a visible 10-minute timer if you can.

Your turn. Build something.

40 minutes · 25 build · 15 peer review

ALL SIX 201 SKILLS, IN PRACTICE

SPEAKER NOTES

SECTION DIVIDER One beat. Then immediately into the assignment slide.

Say: “Module 3. Forty minutes. The first 25 you build. The last 15 you and a partner review what you built. I am circulating the whole time — I will not solve your problems for you, but I will ask the question that gets you unstuck.”

Pick one starter problem — or your own.

OPTION 1**Leave request tracker**

Who, dates, type, approval status.

OPTION 2**Training attendance log**

Event, date, attendees, completion.

OPTION 3**Vehicle inspection checklist**

Vehicle ID, date, items, pass/fail.

OPTION 4**Your own section problem**

Get instructor sign-off on scope first.

SPEAKER NOTES

ACTIVITY · 25 MIN Set a visible timer. Tell the room you'll call out the 15- and 5-minute marks.

Say: “Pick one of these four. If you pick Option 4, ping me first — I want to make sure your scope is buildable in 25 minutes. Decompose on paper for two minutes. Then start prompting.”

What ‘done’ looks like: A working core function — even if rough. Not pretty. Not complete. **Working.**

Circulate constantly. Do NOT let any student struggle silently for more than three minutes. Use these scripts:

- Stuck on decomposition: “What is the simplest version that would be useful?”
- Stuck on a prompt: “Read me your prompt out loud.” (They usually catch it themselves.)
- Stuck on an error: “What does the error message say? Copy it exactly and ask AI to explain it.”
- Building too big: “Stop. What is the one core function this needs to do? Build only that first.”

Don’t fix their code. Ask the question that leads them to the answer. The debugging process **is** the lesson.

When errors appear (and they will): Click forward to the “When something breaks” slides — show the room the debugging loop, then send them back to their build. Even better: pick a real error a student is hitting and walk through it on screen.

Time cues: Call “15 minutes left,” “5 minutes left,” and a hard stop at 25.

Errors are not failure. They are the whole job.

The first time something breaks, most people quit. That is the 80% who stop building. Builders expect errors and know how to fix them. You are about to be a builder.

FIRST

Read the error out loud.

Don't guess. Don't skim. Read exactly what the system says, word for word. Most of the “hard” problems explain themselves once you read them.

THEN

Copy it — don't paraphrase.

Copy the exact error message including any codes or line numbers. Paraphrasing strips the information AI needs to diagnose it.

SPEAKER NOTES

WHEN SOMETHING BREAKS · INTERRUPTION SLIDE Click here when you spot the first real error in the room. Stop the build clock briefly. Use this as a teaching moment for the whole class.

Say: “Hold up. Eyes on me for two minutes. Someone just hit an error — that's the most useful moment of the day. Here's what builders do.”

Hammer the framing: “The first time something breaks, most people quit. That is the 80% from Week 1 — the people who stop using AI. Today, every one of you is going to break something. The difference between you and the 80% is what you do next.”

Two-step rule: Read the error out loud. Copy it word-for-word. That's slide 1. Slide 2 covers the debug loop. Slide 3 shows the prompt template.



Three windows. One loop.

STEP 1

Power Apps shows the error.

Read it. Copy it. Don't close the window — you'll need it again.

STEP 2

GenAI.mil diagnoses it.

Paste the error *and* what you were trying to do *and* the relevant code or column setup. Three things, not one.

STEP 3

You apply the fix.

Copy the corrected code back into Power Apps. Test it. If it still breaks, repeat — don't try a fourth different fix.

Different tool, same loop. If you're stuck on CamoGPT, paste the same error into ChatGPT or Gemini for a second opinion (unclassified prompts only). Using multiple tools is a valid strategy.

SPEAKER NOTES

SHOW THE LOOP 60 seconds on this slide.

Say: “Three windows. One loop. Power Apps shows the error. You copy it — not paraphrase, copy — into GenAI.mil. GenAI.mil tells you what's wrong. You apply the fix in Power Apps. That's the whole job.”

Common mistake to call out: “If your fix doesn't work, don't try a fourth different fix. Repeat the loop — copy the new error in and ask again. Random changes make it worse.”

Tool flexibility: “If GenAI.mil is giving you weird answers on a specific error, paste the same error into ChatGPT or Gemini for a second opinion. Just remember — commercial tools are unclassified-only. Don't paste CUI in.”

Transition: “Last slide on this. The shape of a debug prompt.”



A debug prompt has three pieces.

I am building a leave request tracker in Power Apps.

I added this formula to my gallery:

```
Filter(LeaveRequests, Status = "Pending"
      && RequestDate > Today())
```

I am getting this error:

```
"The function 'Filter' has some
  invalid arguments."
```

The RequestDate column is a Date field
in my SharePoint list.

What is wrong and how do I fix it?

- **Context** — what you’re building and where this code lives.
- **Exact code** — copied from your tool, not retyped.
- **Exact error** — the real message, in quotes.
- **Plus relevant setup** — column types, choice values, anything AI can’t see.

If you’ve sent a debug prompt and it didn’t work, ask:
which of these four did I leave out?

SPEAKER NOTES

ANATOMY OF A DEBUG PROMPT 90 seconds on this slide.

Say: “Look at this prompt. It’s four sentences and a code block. Most students send AI a one-line cry for help — **‘my filter doesn’t work’** — and then wonder why AI gives them garbage. Look at the four pieces here.”

Walk through:

- Context: **“I am building a leave request tracker in Power Apps.”**
- Exact code: copied straight from the tool.
- Exact error message: in quotes.
- Setup AI can’t see: **“The RequestDate column is a Date field.”**

Test the room: “If you send a debug prompt and it doesn’t fix it, ask yourself: which of these four did I leave out? Almost always, the answer is the setup AI can’t see — the column types, the choice values, what your data actually looks like.”

Send them back to the build: “OK. Back to your builds. Use this template when you hit your next error. I’m circulating.”

Show your partner what you built — even if it’s rough.

Each person gets ~6 minutes. Demonstrate it. Then your partner walks you through the four checks below before you swap.

CHECK 01**Clear problem statement**

Can the builder explain the problem in one sentence?

CHECK 02**Core function works**

Does the basic version do what it needs to do?

CHECK 03**Evidence of iteration**

Did the builder refine at least once?

CHECK 04**Can explain decisions**

Can the builder say *why* they made the choices they made?

SPEAKER NOTES

PEER REVIEW · 15 MIN Set a visible timer. Pair students yourself — don’t leave it to chance, you’ll get one strong + one weak in each pair on purpose.

Time slices:

- 0–2 min: explain the four checks.
- 2–8 min: Partner A demos, Partner B reviews.
- 8–14 min: Partner B demos, Partner A reviews.
- 14–15 min: Whole-class debrief — one or two standout examples.

Circulate: Listen for constructive feedback. If a reviewer is being too soft (“looks great”), step in and ask the four checks yourself. If a reviewer is being harsh, redirect to specifics.

Watch for: Builders who can’t answer Check 04 (“why did you make that choice?”) usually didn’t decompose — they just copy-pasted what AI gave them. Flag this for Module 4.

Debrief question: “What did you learn from seeing your partner’s approach that you didn’t see in your own?” Get one or two answers. Move on.

Decomposition, applied to your problem.

20 minutes · 10 individual · 8 pair · 2 debrief

THE MAKE-OR-BREAK EXERCISE OF THE DAY

SPEAKER NOTES

SECTION DIVIDER Hold for one beat.

Say: “Module 4. Twenty minutes. This is the most important exercise of the day. The decomposition you walk out with is the assignment you’ll execute before Course 3. Take it seriously.”

Pick one real problem from your section. Decompose it on paper.

Not a fake one. Not a clever one. The annoying recurring one you've been meaning to fix.

- **Problem statement** — one sentence. Concrete enough that a Marine outside your unit could understand it.
- **Core requirements** — what must this do? List them.
- **4-6 subtasks** — with a Human role, an AI role, and the pattern (Centaur or Cyborg) for each.
- **Frontier risks** — what might AI struggle with? Authentication? Integration with a legacy system? Domain-specific calcs?
- **Simplest useful version** — what could you ship in 1-2 hours that would still help?

SPEAKER NOTES

ACTIVITY · 10 MIN INDIVIDUAL Set a visible timer. The worksheet template is in their student companion (or hand out a paper copy if you printed them).

Say: "Pick a real problem from your section. Not a clever one — the annoying recurring one you've been meaning to fix. Decompose it on paper. You have ten minutes. Go."

Walk the room actively. Quality of decomposition predicts quality of build. Watch for these failure modes:

- **Problem too big:** if subtasks list more than 8 items, scope is wrong. Help them narrow to one slice.
- **Problem too vague:** if the statement uses words like "better" or "improved," push for concrete requirements. "What does 'better' mean? Faster? Fewer steps? Less paperwork?"
- **No frontier recognition:** if they don't list any AI limitations, prompt: "What part of this might be hard for AI?"
- **All-AI subtasks:** if every subtask says "AI builds," remind them — the human provides domain knowledge. Where does theirs show up?

Time cues: 5-minute warning. 1-minute warning. Hard stop at 10.

Trade worksheets. Run the four checks.

Four minutes per person. Use a visible timer. Don't fix it for them — ask the question that surfaces the gap.

CLARITY CHECK

Could someone outside the unit understand the problem? If not, push for specifics.

SCOPE CHECK

Is any subtask too big? Should it be broken down further?

FRONTIER CHECK

What did the builder miss? Authentication? Data security? Legacy integration?

MVP CHECK

Is the simplest useful version actually simple? Could you build it in 1–2 hours?

SPEAKER NOTES

PAIR REVIEW · 8 MIN Same pairs as Module 3, or re-pair if the chemistry was off. Visible timer.

Say: “Trade worksheets. Four minutes per person. Use the four checks. Your job isn't to fix it — your job is to ask the question that helps your partner see the gap.”

Intervention signal: If a pair is stuck on clarifying the problem for more than 2 minutes, step in. Ask the builder: “What does success look like? Describe the end state.”

Listen for: Builders who realise mid-review that their MVP isn't actually minimal. Highlight that on debrief — “narrowing scope is the most common ‘aha.’”

Decomposition is a skill. You will get better with reps.

TWO QUESTIONS FOR THE ROOM

- Who changed their decomposition based on partner feedback?
- What did your partner catch that you missed?

THE PATTERN

The first time you decompose anything, you will miss things. That's normal. The second time you decompose the same problem, you will miss fewer. By the tenth time, decomposition is automatic.

By the end of this module, every student should walk out with a one-page decomposition of a real problem they will build before Course 3. If yours is still vague, grab me at the break before Module 5.

SPEAKER NOTES

DEBRIEF · 2 MIN Quick. Hands and one or two examples.

Say: “Show of hands — who changed their decomposition based on what your partner said?” (Most should raise hands. If nobody does, ask why — usually means partners were too soft.)

Take 1-2 examples: “What did your partner catch that you missed?” Pick someone who looked engaged. Don't cold-call someone who looked frustrated.

Drive home: “Decomposition is a skill. You'll get better with reps. The first time you decompose anything, you will miss things. That's normal. By the tenth time, it's automatic.”

Pull aside privately (after class): any student whose worksheet is blank or vague. They're not ready for the assignment — help them scope.

Wrap-up. Then your homework.

10 minutes · recap · assignment · what's next

YOU LEAVE WITH A REAL PLAN

SPEAKER NOTES

SECTION DIVIDER One beat. Energy back up — this is the close.

Say: “Last ten minutes. Three things — quick recap of what you learned, the assignment before Course 3, and a preview of what’s next.”

Four questions. Answer in your head, then check.

1

What is the first thing a builder does — before any tool?

Decompose on paper. Data fields, user actions, simplest useful version.

2

What are the three pieces of a good debug prompt?

Context, exact code, exact error — plus relevant setup AI can't see.

3

When AI gives you the wrong answer, what do you do?

Re-prompt with what was wrong. Don't accept the first output. Don't make random changes.

4

What separates the 20% who keep building from the 80% who quit?

Builders expect errors and debug them. Quitters treat the first error as failure.

SPEAKER NOTES

RECAP • 3 MIN Cover the question. Ask the room. **Then** reveal the answer.

Say: “Four questions. Answer in your head before I show the answer. If you can answer all four, you're ready for Module 4's assignment.”

For each question: Read the question. Pause for 5 seconds (it will feel longer than you think). Take one volunteer answer. Then point to the answer on screen.

If the room is shaky on Q1: Re-state — “Decompose. On paper. Before any tool. If you remember nothing else, remember that.”

Transition: “Here's your assignment.”

Come to Course 3 with four things.

- **One.** A working (or partially working) prototype of the problem you decomposed today.
- **Two.** Notes on what worked — prompts that landed, features that came together fast.
- **Three.** Notes on what didn't — errors you hit, features you cut, dead ends.
- **Four.** One failure case worth sharing — something AI got wrong that other builders should know about.

Time budget: 2-4 hours over the next week. Not a finished product. A real attempt. Office hours and the #builder-orientation Teams channel are open if you get stuck.

SPEAKER NOTES

ASSIGNMENT Slow down here. People will tune out at the end of class — this is what they're paid to remember.

Say: "Four things. One: a working or partially working prototype of the problem you just decomposed. Two: notes on what worked. Three: notes on what didn't. Four: one failure case worth sharing — something AI got wrong that other builders should know about."

Set realistic expectations: "Two to four hours total over the week. This is not a finished product. It's a real attempt. If your prototype only does the simplest version of the simplest version, that's fine — show up with it anyway."

Why the failure case matters: "Course 3 starts with failure sharing. Every failure case you bring saves another builder an hour of confusion. That's how the unit's frontier map gets built."

Support resources: Office hours (Wed 1500-1600), #builder-orientation Teams channel, ai-builders@1stbn99thmar.mil. State the times out loud.

If anyone looks lost: Note them. Catch them after class for a 5-minute scoping conversation.

WHAT'S NEXT

Week 3 — Platform Training.

Four hours. Three complete tools. You practice both Centaur and Cyborg patterns and walk out able to deploy under the EDD SOP.

Setup & Review	15 MIN
Build #1 — Centaur Mode	60 MIN
Failure Sharing (during break)	15 MIN
Build #2 — Cyborg Mode	60 MIN
Build #3 — Your Problem	60 MIN
Frontier Map Update & Wrap	15 MIN

BRING YOUR PROTOTYPE · BRING YOUR FAILURE CASE · BRING YOUR LAPTOP

SEE YOU AT COURSE 3

SPEAKER NOTES

WEEK 3 PREVIEW 60 seconds. Set the expectation, then close.

Say: “Course 3 is Platform Training. Four hours. You’ll build three complete tools — one in Centaur mode with explicit handoffs, one in Cyborg mode with continuous AI back-and-forth, and one of your own.”

Failure sharing reminder: “The failure-sharing block during the break is not optional. It’s one of the most valuable 15 minutes in the whole program. That’s why I asked you to bring a failure case.”

Logistics: Confirm the date and time of Course 3. Remind them: bring the prototype on a laptop, thumb drive, or screenshots.

Closing message: “You are not expected to be experts after two hours. You are expected to try. Every builder hits errors. The difference is that builders debug instead of quit. Decomposition is the skill that makes everything else work. Help each other. Share failures openly. That’s how we all get better.”

End on time. Stop the screen share. Stay on the call for five more minutes for one-on-one questions.